



PROMPT ENGINEERING IN BUSINESS

Term Project Report

December 06, 2024

Abibat Oki

abibatoki@cmail.carleton.ca

Introduction

Grading student assignments is an essential yet time-intensive task in academic institutions. As class sizes grow and educators manage heavier workloads, the need for scalable, consistent, and efficient grading solutions becomes evident. To address this, the Retrieval-Augmented Generation (RAG) Grading Agent leverages Large Language Models (LLMs) and retrieval mechanisms to automate the grading process. This report outlines the design, architecture, proof of concept, and future prospects of this agent, with a focus on its deployment in an academic setting.

Problem Context and Motivation

The Problem

Grading assignments in academic institutions is a slow, inconsistent process, especially for classes with 30 or more students. Manually evaluating submissions leads to delays, variability in feedback, and limits scalability. Without automation, professors spend hours on repetitive tasks, reducing their ability to focus on teaching. This inefficient workflow creates a significant challenge for educators, highlighting the urgent need for a faster, consistent, and scalable grading solution.

Motivation

The motivation for this project was to explore how Retrieval-Augmented Generation (RAG) agents, a growing AI approach, can transform the grading process in academic settings. The goal was threefold: to automate grading, freeing educators to focus on teaching and mentoring; to ensure standardized, rubric-based feedback for all students; and to save time, enabling faster feedback cycles that directly enhance student learning.

Solution Design

Overview

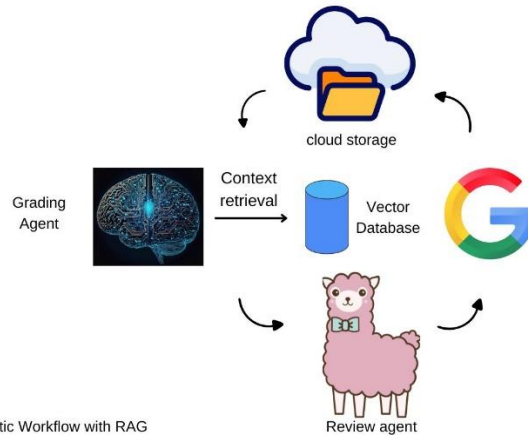


Fig 1.0 Static Workflow with RAG

The RAG grading agent is designed to:

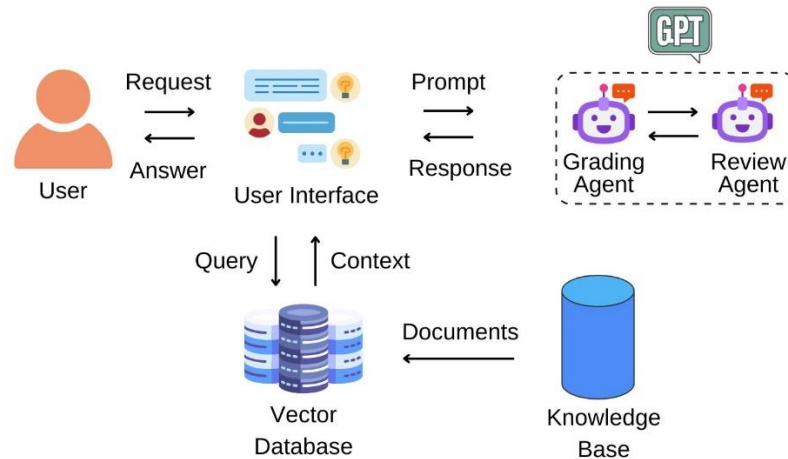
- Automate grading using LLMs.
- Leverage a domain knowledge base for context retrieval, enhancing grading quality.
- Provide clear, rubric-aligned feedback for every submission.

System Architecture

The agent comprises the following key components:

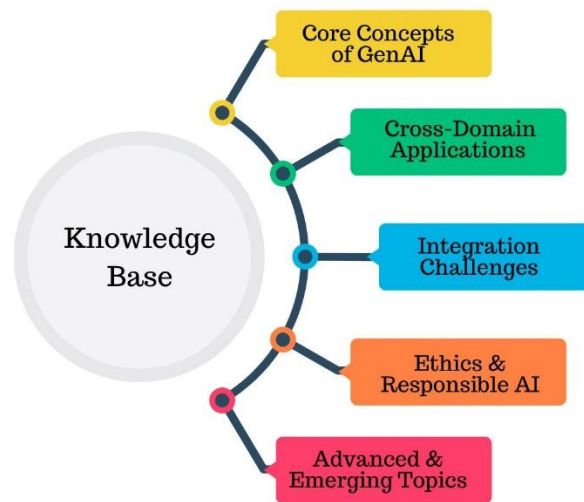
1. Processing Script: Prepares and batches student submissions for analysis.
2. Grading Agent: Uses predefined rubrics and assignment questions to evaluate submissions. Retrieved domain-specific context augments the grading process.
3. RAG Framework: Employs vector databases for retrieving relevant contextual information, ensuring a deeper and more informed analysis.
4. Reviewer Agent: Reviews the grading feedback, refining it for clarity, specificity, and strict adherence to rubric criteria.

5. Output Module: Exports feedback in structured formats such as JSON or CSV and distributes it to students using automated workflows.
6. Error Handling and Logging: Tracks system performance and resolves issues like failed file processing or incomplete retrieval.



Domain Knowledge Base

A core feature of the RAG grading agent is its ability to retrieve domain-relevant knowledge. The knowledge base consists of 90 documents divided into five categories:



1. Core Concepts of Generative AI and LLMs

- Books, research papers, and tutorials on the Transformer architecture, embeddings, and RAG systems.

2. Cross-Domain Applications of Generative AI

- Case studies in retail, finance, healthcare, education, entertainment, and sustainability.

3. Business and IT Integration Challenges

- White papers and industry reports on integrating generative AI into organizations.

4. Ethics and Responsible AI

- Frameworks and case studies addressing bias, transparency, and compliance issues.

5. Advanced and Emerging Topics

- Research on diffusion models, multimodal AI, and advanced prompt engineering techniques.

Process and Workflow

The RAG grading agent follows a structured workflow that ensures high-quality grading outputs:

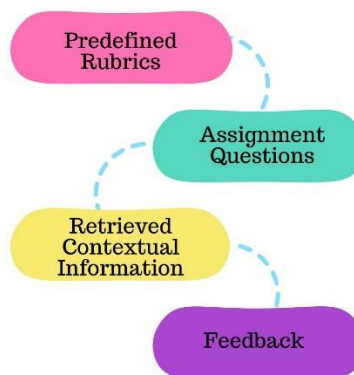
1. Data Acquisition:

- Assignments are uploaded in PDF format.
- Text is extracted using PyMuPDF and processed into clean, analyzable input.
- Domain knowledge is segmented into manageable chunks (~1,000 tokens) using recursive character text splitter for efficient retrieval.

2. Grading:

The agent evaluates submissions by combining:

- Predefined rubrics, which include detailed grading criteria for the assignment.
- Assignment questions, which set the scope for evaluation.
- Retrieved contextual information from the domain knowledge base.
- Feedback is generated through a custom grading prompt optimized for alignment with the rubric



3. Review and Refinement:

A reviewer agent further refines feedback, ensuring:

- Clarity of comments.
- Specificity in addressing strengths and weaknesses.
- Adherence to rubric criteria through a prompt.

4. **Output Generation:**

Final feedback is saved and distributed to students.

Proof of Concept

Data Handling/Preparation

The data preparation process ensures that all inputs are clean, structured, and ready for processing by the RAG grading agent. The pipeline involves the following steps:

1. Text Extraction: Assignment files, typically in formats like PDF or Word, are processed using PyMuPDF to extract text accurately.
2. Text Cleaning: Extracted text is cleaned to remove any artifacts, such as formatting errors or noise from scanned documents, ensuring readability.
3. Text Structuring: The agent uses predefined prompts and rubric templates to extract and evaluate relevant sections directly from the submission

To ensure correct rubric matching across varied rubrics

- A pre-processing script will be used to identify and tag rubric criteria in the input data.
- Rubric templates will be modular, allowing for seamless customization and alignment to different grading standards.
- Prompt refinement for clarity and rubric alignment will involve iterative testing, where feedback outputs are validated by the user (e.g., educators) to ensure they meet rubric-specific expectations.

Data Alignment with Components

- Assignments: Serve as input for the grading agent to evaluate against the rubric.

- **Rubric and Prompts:** Predefined templates tailored to different rubrics ensure flexibility and relevance.
- **Domain Knowledge:** Context retrieved from the knowledge base enhances grading precision by providing additional references for complex or technical assignments.

Roles and Responsibilities

- **Initial Setup:** A data engineer or technical support staff will configure and test the data preparation pipeline, ensuring it matches the institution's requirements.
- **Ongoing Operations:** Educators or teaching assistants will manage rubric uploads and oversee data input for new assignments. They will also perform prompt refinement, assisted by example outputs, to ensure consistent feedback generation for each rubric type.
- **Validation:** Educators will periodically validate feedback to fine-tune rubric matching and adjust prompts for alignment with specific course requirements.

This structured approach ensures that the data handling process is comprehensive, adaptable to different rubrics, and seamlessly integrates into the academic workflow.

LLMs and Prompt Engineering

LLM Model: The grading agent utilizes OpenAI's GPT-3.5 to generate feedback for student submissions, while the reviewer uses GPT-4 for refining feedback, leveraging its advanced reasoning capabilities. Both models have been selected for their respective strengths in these tasks; however, they can be customized or replaced with alternative models based on user needs and preferences, ensuring flexibility in deployment.

Two custom-designed prompts were developed:

1. **Grading Prompt:** Used to evaluate submission content against rubric criteria such as content relevance, clarity, and adherence to instructions. It incorporates key inputs like:
 - The assignment question.

- The rubric criteria.
- Retrieved contextual knowledge.

Grading Prompt/Rubric

You are a professor of technology innovation management. Your task is to grade student assignment submissions. Use the rubric below and the provided assignment question and student submission to evaluate the assignment. Assign scores from 1 to 5 for each criterion based on the levels of detail, alignment with objectives, and overall completion. For each criterion, provide detailed feedback explaining the score.

Assignment Question: {assignment_question}

Submission Content: {assignment_content}

Rubric: {rubric}

1. Selected Solution (1-5 points)

- Level 4 (5 points): The solution uses an LLM or an LLM-based platform as a core component essential to the business context.
- Level 3 (4 points): The solution uses an LLM-based component, but its role within the business context lacks clarity.
- Level 2 (3 points): A general-purpose tool (e.g., ChatGPT, Gemini, Copilot) is used, which doesn't align with the assignment objective.
- Level 1 (1 point): No LLM-based solution is identified.
- Feedback Requirements: Identify specific strengths of the solution and any areas of misalignment or areas where the solution falls short. For lower scores, suggest potential improvements to clarify or align the chosen solution with the assignment's objectives.
- Clarification: "Core component essential to the business context" means the LLM-based component provides a unique function integral to achieving the assignment's objectives and would not be easily accomplished with alternative technologies...

Reviewer Prompt: Focused on refining the grading feedback for clarity and specificity ensuring alignment with the rubric while suggesting actionable improvements.

Reviewer Prompt

You are an expert in prompt engineering and feedback optimization. Your task is to review feedback provided by a grading agent to ensure it aligns with the grading rubric. You will refine the feedback for clarity, specificity, and adherence to the rubric, and return a final version in the exact format provided below.

Instructions:

1. Validation Against Rubric:

For Each Criterion:

- Evaluate the grading agent's feedback (provided below) to determine if it aligns with the rubric's requirements:

{feedback}

- Retain the original scores unless there is a clear and justifiable error.

- Check for inconsistencies, inaccuracies, or omissions in the comments and scores.

- Adjust scores or comments if the feedback does not fully or accurately reflect the rubric's criteria:

{rubric}

2. Provide Reasoning:

- For each adjustment (or if no adjustments are made), include a brief explanation of why the score and feedback are correct or why changes were necessary.

- Ensure that all changes are rooted in the rubric's standards.

3. Refine Feedback:

- Enhance the clarity and depth of the feedback.

- Ensure all strengths and weaknesses are identified and tied directly to rubric criteria.

- Provide actionable and specific suggestions for improvement, including examples where applicable...

The agent was validated using a dataset of 30 student submissions to assess the accuracy of feedback generated. Due to time constraints and the proof-of-concept nature of this project, the feedback outputs were not compared with human graders. Instead, the accuracy was assessed based on the developer's experience and domain knowledge, focusing on the alignment of feedback with rubric criteria and assignment instructions.

Each feedback instance was reviewed to ensure it addressed specific rubric components and provided actionable and relevant evaluations. While no quantitative assessments of technical outputs were conducted, qualitative checks confirmed that the feedback was accurate and rubric-aligned, demonstrating the system's potential to generate reliable results within a controlled academic setting.

Expected vs. Actual Outcomes

The system exceeded expectations in terms of speed and consistency. It processed 30 assignments in under 16 minutes, compared to manual grading, which typically takes hours. Feedback accuracy was high, with occasional deviations due to ambiguity in rare(exceptional) cases. These deviations were identified and addressed through iterative prompt refinement.

Organizational Impact

- **Ease of Use:** The system's modular design, with customizable rubric and prompt templates, will allow educators to adapt it to specific courses effortlessly.
- **Collaboration Efficiency:** By automating grading, educators could allocate more time to teaching, mentoring, and improving course content.

Risk Assessment

Over-Reliance on LLMs

A significant risk in using the RAG grading agent lies in over-reliance on Large Language Models (LLMs). These models may misinterpret vague or poorly structured inputs, potentially producing irrelevant or inaccurate feedback. This risk is particularly high when the assignment submissions deviate from the expected format or include ambiguous language. To mitigate this, iterative prompt refinement was employed to ensure that the

prompts are clear, aligned with the rubric, and capable of guiding the LLM to generate relevant outputs. Additionally, human validation was incorporated into the process to periodically review the feedback for quality assurance, ensuring that any errors or inconsistencies are identified and addressed promptly.

Data Security

Handling sensitive student data introduces risks related to privacy and ethical concerns. Submissions often contain personal identifiers and academic work, making compliance with privacy standards such as GDPR or FERPA essential. To address this, data anonymization techniques were implemented, stripping submissions of any personally identifiable information before processing. Furthermore, the system was designed to adhere to strict ethical guidelines, ensuring that data is handled securely and responsibly. By prioritizing privacy at every stage of the pipeline, the RAG grading agent mitigates risks associated with data breaches and unauthorized access.

Scalability

As the number of submissions grows, the system faces scalability challenges that could lead to slowdowns or failures. Larger datasets require greater computational resources, and inefficient retrieval mechanisms can bottleneck performance. To tackle this, the retrieval process was optimized through enhanced vector database indexing and chunking strategies, enabling faster and more efficient access to relevant knowledge. Additionally, distributed processing methods were employed to ensure the system can handle large volumes of data without compromising performance. These measures make the RAG grading agent more robust and capable of scaling to meet the needs of larger classes or institutions.

Risk Ranking Based on Impact and Likelihood

Data Security (High Impact, High Likelihood)

Data breaches or mishandling sensitive student information can result in serious ethical and legal repercussions, making this the most critical risk. The likelihood of this risk is

significant due to the volume of personal data processed and the potential vulnerabilities in data handling.

Scalability (Medium Impact, Medium Likelihood)

Scalability challenges can slow down the system, particularly during peak usage periods or with large datasets. While the system is designed to handle these situations, limitations in computational resources could still pose a medium likelihood of occurrence with a moderate impact on grading efficiency.

Over-Reliance on LLMs (Medium Impact, Low Likelihood)

Errors from LLMs interpreting vague or ambiguous submissions can lead to irrelevant feedback. However, this risk is less likely in a live environment with well-structured assignment prompts and the safeguards of iterative prompt refinement and human validation.

Contingency Plan for Significant Failure Points

Data Security

Preventive Measures:

- Implement strong encryption for all data in transit and at rest.
- Enforce strict access controls to limit data exposure to authorized personnel only.
- Conduct regular audits to identify vulnerabilities in data handling and storage.

Contingency Actions:

- Immediately disable affected systems in the event of a breach.
- Notify stakeholders and affected parties as required by privacy regulations (e.g., GDPR, FERPA).
- Work with cybersecurity experts to identify and patch vulnerabilities.
- Restore operations from secure backups to minimize downtime.

Scalability

Preventive Measures:

- Continuously monitor system performance metrics to preemptively identify bottlenecks.
- Use cloud-based solutions to dynamically scale computational resources based on workload.

Contingency Actions:

- Temporarily reduce the batch size for grading assignments to manage system load.
- Engage additional resources for distributed processing to maintain service levels during high-demand periods.
- Notify users of potential delays and provide realistic timelines for completion.

Over-Reliance on LLMs

Preventive Measures:

- Provide clear and structured instructions to students for assignment submissions to minimize ambiguities.
- Regularly test prompts and feedback outputs to ensure alignment with rubrics and desired outcomes.

Contingency Actions:

- Introduce a manual review mechanism for flagged assignments where LLM outputs appear inconsistent or unclear.
- Assign an educator or teaching assistant to validate critical feedback points.
- Refine prompts dynamically in response to common submission patterns or errors identified in live use.

Key Insights and Challenges

Challenges Encountered

One of the key challenges during the project was prompt refinement. Early iterations of the grading prompts generated feedback that lacked specificity, often failing to provide actionable or detailed evaluations for certain submissions. To address this, the prompts were iteratively improved by refining the input templates and feedback formatting. This iterative process involved testing the prompts with sample assignments, analyzing the outputs for alignment with the rubric, and making adjustments to ensure that the feedback became more precise and tailored to each criterion.

Another challenge was related to knowledge retrieval. Initial retrieval attempts often produced inconsistent results, as the system lacked a robust knowledge base to provide context for grading complex topics. This issue was resolved by creating a comprehensive knowledge base, populated with relevant documents spanning all necessary topics. The enhanced knowledge base improved retrieval accuracy, enabling the grading agent to generate well-informed feedback that aligned closely with the rubric and assignment requirements.

Insights

Batch Processing

The implementation of batch processing has significantly streamlined the grading workflow, allowing multiple submissions to be graded simultaneously. This capability reduces the manual workload for educators, who would otherwise have to process each submission individually.

Domain Knowledge Integration

The integration of a robust domain knowledge base enhances the grading agent's ability to provide nuanced and context-aware feedback. By retrieving relevant information specific to the assignment topic, the system generates feedback that is richer and more aligned with the academic requirements. This integration proves especially useful for

complex or technical assignments where additional context can improve the depth and relevance of the evaluation.

Impact on Grading Quality

Together, batch processing and domain knowledge integration significantly improve the quality and efficiency of grading. Batch processing ensures speed and scalability, while domain knowledge integration enriches the content of feedback, making it more tailored to each submission. This dual benefit demonstrates the value of combining automation with contextual intelligence, transforming grading into a more streamlined and effective process.

Limitations and Future Scope

Limitations

The RAG grading agent currently has limited functionality in handling multimedia submissions such as videos, diagrams, or audio files, restricting its application to text-based assignments. Additionally, the system relies heavily on well-structured rubrics and clear assignment questions to produce accurate and relevant feedback. This dependency could limit its effectiveness in scenarios where grading criteria or instructions are ambiguous.

Future Scope

1. Multimedia Support

Expanding the agent's capabilities to grade non-textual submissions, such as videos, diagrams, or presentations, would make it applicable to a broader range of assignments. This enhancement could include integrating multimodal AI systems capable of processing and evaluating various media formats.

2. Ethics and Bias Management

Developing mechanisms to detect and mitigate biases in grading feedback would ensure fairness and transparency. This could involve incorporating ethical AI frameworks and

tools to identify biased language or patterns in feedback and align outputs with inclusive practices.

3. Broader Application

The system's potential extends beyond academia to areas like corporate training, employee performance evaluations, and professional certification grading. By tailoring the agent for these use cases, organizations could benefit from consistent, scalable, and automated evaluation processes.

Conclusion

The RAG grading agent offers a transformative solution to the challenges of manual grading in academia. By automating the process and integrating domain knowledge, it delivers consistent, efficient, and rubric-aligned feedback. With further development, this system has the potential to serve a variety of applications beyond education, paving the way for AI-driven innovation in evaluation and feedback processes.